Pflichtenheft – Template Service (template-service)

Projekt:	Multi-Tenant SaaS-Plattform
Service:	template-service
Version:	1.2
Stand:	28.02.26
Technologie:	Java 21, Spring Boot 4.x, Maven, MariaDB/MySQL, Flyway, Docker, OpenAPI/Swagger, Spring Security (OAuth2 Resource Server)

1. Zielsetzung

Der template-service verwaltet, versioniert und rendert Templates (TEXT/HTML) für verschiedene Kommunikationskanäle (z. B. E-Mail, In-App/Inbox). Der Service stellt eine REST-API bereit, über die Templates erstellt, freigegeben (Approval), gelistet und zur Laufzeit gerendert werden.

Der Service unterstützt:

- Plattform-Templates (**GLOBAL**) für den Plattformbetreiber.
- Mandanten-Templates (**TENANT**) je Mandant.
- Versionierung mit Statusmodell (DRAFT/APPROVED/DEPRECATED/ARCHIVED).
- Rendering mit Placeholdern `{{key}}`, Missing-Key-Verhalten und HTML-Escaping.
- Preview/Validate für Admin-UI und Build-/QA-Checks.
- Absicherung über IAM (JWT Bearer Token mit Scopes).
- Mandantentrennung über `tenant_id` aus dem Token.

Abgrenzung:

- Kein Versand/Delivery von Nachrichten (liegt in Messaging/Delivery-Services).
- Keine Erstellung oder Persistenz von Inbox-Nachrichteninstanzen (liegt im Inbox/Messaging-Service).
- Keine Token-Ausstellung (liegt im auth-service).

2. Systemkontext und Service-Verbindungen (namentlich)

2.1 Inbound (wer ruft `template-service` auf)

`template-service` ist ein Backend-Service, der von folgenden Gegenstellen genutzt wird:

1. **Portal/Frontend (Admin-UI)** → `template-service`
   Zweck: Template-Verwaltung (CRUD), Preview/Validate sowie Katalog-Ansichten für GLOBAL/TENANT Templates.
2. **`messaging-service`** → `template-service`
   Zweck: produktives Rendering von Templates (Text/HTML) im Outbound-Fluss (z. B. E-Mail/SMS/IN_APP), i. d. R. über `/api/v1/template/render`.
3. **`communication-service`** → `template-service`
   Zweck: Rendering im Rahmen von Sendeaufträgen (templateRef + payloadJson), sobald die Template-Integration aktiv genutzt wird.
4. **`document-service`** → `template-service`
   Zweck: Erstellung/Komposition von Dokumentinhalten auf Basis von Templates (z. B. für spätere Dokument-Generierung).

5. **Batch-/QA-Tools** → `template-service`
   Zweck: Validierung/Regression-Checks (z. B. Syntax/Placeholder-Konsistenz) im CI/CD bzw. Testbetrieb.

2.2 Outbound (wen ruft `template-service` auf)

- **Keine fachlichen Downstream-Aufrufe.**
- **Technische Abhängigkeit:** `template-service` lädt die Public Keys (JWKS) zur JWT-Signaturprüfung über die konfigurierte JWKS-/Issuer-Quelle; in der Plattform ist dies typischerweise der **`auth-service`** (JWKS Endpoint).

3. REST-API – Übersicht

3.1 Basis-URL, Versionierung, Content Types

- Server-Port: **8104**
- `server.servlet.context-path=/api/v1`
- Controller Base Path: `/template`

Effektive Basis-URL:

- **`/api/v1/template`**

Content Types:

- Request/Response: `application/json`
- Fehler: `application/json` (ErrorDTO)

3.2 Fachliche Endpunkte

Rendering / Preview / Validate

1. **POST** `/api/v1/template/render`
   Rendert ausschließlich die freigegebene (APPROVED) Version der aufgelösten Template-Familie.
2. **POST** `/api/v1/template/preview`
   Rendert eine Version unabhängig vom Status für Vorschauzwecke (z. B. DRAFT). Für produktive Nutzung wird `/render` verwendet.
3. **POST** `/api/v1/template/validate`
   Validiert Template-Syntax und Placeholder-Konsistenz. Optional simuliert die Validierung das Missing-Key-Verhalten mit Beispielvariablen.

Catalog

4. **GET** `/api/v1/template/catalog`
   Query:

- `scope` (GLOBAL|TENANT)
- `channel`
- `locale`
- `page`, `size`, `sort`

5. **GET** `/api/v1/template/catalog/{templateId}/approved-versions`
   Liefert freigegebene Versionen einer Familie.

Template-Familien

6. **POST** `/api/v1/template/families`
   Erstellt eine Template-Familie.
7. **GET** `/api/v1/template/families`
   Liefert paginierte Liste aller für den Aufrufer sichtbaren Familien:

- GLOBAL-Familien (`owner_tenant_id="__GLOBAL__"`)
- TENANT-Familien des aktuellen Tenants (`owner_tenant_id = tenant_id aus JWT`)

8. **GET** `/api/v1/template/families/{templateId}`
   Lädt eine Familie per ID (nur wenn sichtbar gemäß Tenant-Regeln).
9. **PUT** `/api/v1/template/families/{templateId}`
   Aktualisiert Metadaten einer Familie (nur innerhalb der erlaubten Scope-/Tenant-Grenzen).
10. **DELETE** `/api/v1/template/families/{templateId}`
    Löscht eine Familie inklusive zugehöriger Versionen.

Versionen

11. **POST** `/api/v1/template/families/{templateId}/versions`
    Erstellt eine neue Version als DRAFT.
12. **GET** `/api/v1/template/families/{templateId}/versions`
    Listet Versionen der Familie.
13. **GET** `/api/v1/template/families/{templateId}/versions/{versionNo}`
    Lädt eine Version.
14. **PUT** `/api/v1/template/families/{templateId}/versions/{versionNo}`
    Versions-Inhalte werden nicht in-place editiert. Dieser Endpoint liefert einen Konflikt (Immutable-Versionen).
15. **DELETE** `/api/v1/template/families/{templateId}/versions/{versionNo}`
    Löscht eine Version.

16. **POST** `/api/v1/template/families/{templateId}/versions/{versionNo}/approve`
    Freigabe einer Version: Version wird APPROVED, zuvor freigegebene Version der Familie wird DEPRECATED.

3.3 Doku/Ops-Endpunkte

- Swagger UI: `/api/v1/swagger-ui.html`
- OpenAPI JSON: `/api/v1/v3/api-docs`
- Health: `/api/v1/actuator/health`

4. Funktionale Anforderungen

4.1 Begriffe und Grundmodell

- **TemplateFamily**: Identität eines Templates über fachliche Metadaten und Ownership (Scope + OwnerTenantId + TemplateKey + Channel + Locale).
- **TemplateVersion**: Konkrete Version einer Familie, wird nach Erstellung nicht in-place verändert (Immutable).
- **Scope**:
  - `GLOBAL`: Plattformbetreiber
  - `TENANT`: Mandant
- **OwnerTenantId**:
  - GLOBAL: "`__GLOBAL__`"
  - TENANT: `tenant_id` aus JWT
- **RenderTarget**: `TEXT` oder `HTML`
- **MissingKeyPolicy**:
  - `FAIL`: fehlende Keys führen zu 422
  - `KEEP_TOKEN`: Placeholder bleibt stehen
  - `EMPTY`: Placeholder wird leer ersetzt

4.2 Template-Familien (TemplateFamily)

Eine TemplateFamily enthält:

- `templateId` (UUID)
- `scope` (GLOBAL|TENANT)
- `ownerTenantId` (aus Token abgeleitet, siehe Abschnitt 5)
- `templateKey` (max 120)
- `channel` (max 40)
- `locale` (max 16)
- `category` (max 80, Gruppierung)

- `activeApprovedVersion` (Integer, optional)
- Audit-Felder (createdAt/by, updatedAt/by)

Regeln:

- Pro Business-Key existiert genau eine TemplateFamily. Der Business-Key ist: **(scope, ownerTenantId, templateKey, channel, locale)**
  `category` dient der Gruppierung und ändert die fachliche Identität nicht.
- Für GLOBAL werden Familien ausschließlich durch Plattform-Administratoren verwaltet.
- Für TENANT werden Familien ausschließlich im eigenen Tenant verwaltet.

4.3 Template-Versionen (TemplateVersion)

Eine TemplateVersion enthält:

- `versionId` (UUID)
- `templateId` (FK)
- `versionNo` (Integer, monoton steigend)
- `status` (DRAFT|APPROVED|DEPRECATED|ARCHIVED)
- `renderTarget` (TEXT|HTML)
- `subjectTpl` (optional)
- `bodyTpl` (Pflicht)
- `placeholders` (automatisch erkannte Placeholder als JSON)
- Audit-Felder

Regeln:

- Erstellung einer Version erzeugt eine neue `versionNo` und setzt `status=DRAFT`.
- Inhalte einer Version werden nicht in-place geändert. Änderungen erfolgen durch Erzeugung einer neuen Version.
- Approve-Regel:
  - Zielversion wird `APPROVED`
  - vorherige APPROVED-Version der Familie wird `DEPRECATED`
  - `activeApprovedVersion` in der Familie zeigt auf die aktuelle APPROVED-Version

4.4 Rendering, Preview, Validierung

Rendering (`/render`)

- Auflösung der TemplateFamily erfolgt über:
  - bevorzugt Business-Key (scope + templateKey + channel + locale)
  - alternativ `templateId` (Legacy-Kompatibilität)

- Es wird die APPROVED-Version gerendert.
- `versionNo` im Request wird ignoriert (APPROVED wird verwendet).

Preview (`/preview`)

- Rendering unabhängig vom Status, für Vorschau.
- Wenn `versionNo` angegeben ist, wird genau diese Version gerendert.
- Wenn `versionNo` fehlt, wird die neueste Version gerendert (höchste `versionNo`)

Validierung (`/validate`)

- Placeholder werden aus `subjectTpl` und `bodyTpl` erkannt (`{{ key }}`).
- Optional kann eine deklarierte Placeholder-Liste angegeben werden; die Validierung vergleicht “deklarierte” vs. “erkannte” Placeholder.
- Optional kann mit `variables` und `missingKeyPolicy` eine Rendering-Simulation erfolgen.

4.5 Locale-Auflösung und Fallback

Locale-Fallback-Reihenfolge:

1. Exakter Match (z. B. `de-DE`)
2. Sprach-Fallback (z. B. `de`)
3. Default-Locale aus Konfiguration (`TEMPLATE_DEFAULT_LOCALE`, Default `en-GB`)

4.6 Governance-Regeln (No Shadowing, Reserved Keys)

- Reserved Namespaces für TemplateKeys: `platform.*` und `system.*`
  Diese Keys werden nur für GLOBAL akzeptiert.
- No Shadowing:
  - Wenn eine GLOBAL-Familie mit identischem `(templateKey, channel, locale)` existiert, wird die Erstellung einer TENANT-Familie mit identischem Schlüssel abgewiesen.
  - Ziel: GLOBAL bleibt eindeutig und kann nicht tenant-seitig überlagert werden.

4.7 Events (Domain Events)

Der Service erzeugt Domain Events (publishing ist konfigurierbar):

- TemplateFamilyCreated
- TemplateFamilyUpdated
- TemplateVersionCreated
- TemplateVersionApproved
- TemplateVersionDeprecated
- TemplateVersionArchived

Die Events enthalten mindestens:

- `eventType`, `occurredAt`
- `tenantId`
- `scope`
- `templateId`, optional `versionNo`
- `actorSub` (JWT `sub`)

5. Security, IAM und Mandantentrennung (Multi-Tenant)

5.1 JWT Token Contract (Abgleich auth-service)

Der Service arbeitet als OAuth2 Resource Server (Bearer Token). Token werden vom auth-service ausgestellt.

Erwartete Claims im JWT:

- `iss` (Issuer)
- `sub` (Subject; UserId oder ServiceName)
- `aud` (Audience-Liste; enthält `template-service`)
- `iat`, `exp`
- `jti`
- `tenant_id`
- `scp` (Liste der Scopes)

Optional:

- `subject_type` (`USER` oder `SERVICE`)
- `auth_time`
- `sid`, `amr`

Validierungen:

- Signaturprüfung (JWKS oder Issuer)
- Zeitvalidierung (`exp`) mit Clock-Skew aus Konfiguration
- Audience-Check: `aud` enthält `template-service`
- `tenant_id` ist vorhanden und nicht leer

5.2 Scopes und Autorisierung je Endpoint

Scopes (IAM):

- `template:read`
- `template:admin`
- `template:global:admin`

Endpoint-Zuordnung:

- `POST /render` → `template:read`
- `POST /preview` → `template:read` oder `template:admin`
- `POST /validate` → `template:admin`
- `GET /catalog*` und `GET /families*` und `GET /versions*` → `template:read`
- `POST/PUT/DELETE /families*` → `template:admin` oder `template:global:admin` (Scope passend zum Scope des betroffenen Objekts)
- `POST/DELETE/approve /versions*` → `template:admin` oder `template:global:admin` (Scope passend zum Scope des betroffenen Objekts)

5.3 Tenant-Isolation-Regeln (inkl. Non-Leak 404)

Tenant-Kontext:

- `tenant_id` aus JWT ist die einzige Quelle für Tenant-Kontext.

Ownership-Regeln:

- Bei `scope=TENANT` werden alle Datenzugriffe (lesen, listen, rendern, ändern, löschen) auf `ownerTenantId = tenant_id` beschränkt.
- Bei `scope=GLOBAL` werden alle Datenzugriffe auf `ownerTenantId="__GLOBAL__"` beschränkt.

Non-Leak Verhalten:

- Wenn ein Objekt außerhalb des sichtbaren Tenant-/Scope-Kontexts liegt, antwortet der Service mit **404** (nicht mit 403), um keine Existenzinformation preiszugeben.

Request-Felder, die keine Steuerungswirkung haben:

- `ownerTenantId` aus Request wird nicht als Steuerungsfeld verwendet:
  - Bei TENANT wird `ownerTenantId` intern auf `tenant_id` gesetzt.
  - Bei GLOBAL wird `ownerTenantId` intern auf `"__GLOBAL__"` gesetzt.
- Audit-User in Requests (`updatedBy`) wird nicht als Steuerungsfeld verwendet; Audit stammt aus JWT `sub`.

5.4 Inbox-Kontext (Tenant-Grenze)

Inbox-Nachrichteninstanzen (die tatsächlichen Inbox-Objekte) werden tenant-scharf im Inbox/Messaging-Service gespeichert. Der template-service liefert nur gerenderte Inhalte.

Für Templates, die für Inbox/IN_APP verwendet werden, gelten dieselben Tenant-Isolation-Regeln wie für alle Templates:

- TENANT-Templates werden ausschließlich innerhalb des Tenants gerendert und ausgeliefert.
- Ein Tenant erhält keine Render-Ergebnisse von TENANT-Templates anderer Tenants.

6. Datenmodell und Persistenz

6.1 Tabellen und Felder

`template_family`

- `template_id` VARCHAR(36) PK
- `scope` VARCHAR(16) CHECK (GLOBAL, TENANT)
- `owner_tenant_id` VARCHAR(64)
- `template_key` VARCHAR(120)
- `channel` VARCHAR(40)
- `locale` VARCHAR(16)
- `category` VARCHAR(80)
- `active_approved_version` INT
- Audit: `created_at/by`, `updated_at/by`

`template_version`

- `version_id` VARCHAR(36) PK
- `template_id` VARCHAR(36) FK → `template_family(template_id)` ON DELETE CASCADE
- `version_no` INT
- `status` VARCHAR(16) CHECK (DRAFT, APPROVED, DEPRECATED, ARCHIVED)
- `render_target` VARCHAR(16) CHECK (TEXT, HTML)
- `subject_tpl` TEXT NULL
- `body_tpl` LONGTEXT NOT NULL
- `placeholders` JSON NULL
- Audit: `created_at/by`, `updated_at/by`

6.2 Constraints, Indizes, Migrationen

- Flyway Migrationen sind aktiviert.
- Indizes:
  - `idx_template_family_owner` (owner_tenant_id)
  - `idx_template_family_lookup` (scope, template_key, channel, locale)
  - `idx_template_family_category` (category)
  - `idx_template_version_template_status` (template_id, status)
  - `idx_template_version_render_target` (render_target)

7. Konfiguration und Umgebungsvariablen (Tabelle)

Die folgenden Umgebungsvariablen werden vom template-service ausgewertet:

Variable	Default	Beispiel	Beschreibung
`TEMPLATEPORT`	`8104`	`8104`	HTTP-Port des Services (`server.port`).
`TEMPLATE_DB_HOST`	`localhost`	`mariadb`	DB Hostname.
`TEMPLATE_DB_PORT`	`3306`	`3306`	DB Port.
`TEMPLATE_DB_NAME`	`template`	`template`	Datenbankname.
`TEMPLATE_DB_USER`	`root`	`root`	DB Benutzer.
`TEMPLATE_DB_PASSWORD`	*(leer)*	`secret`	DB Passwort.
`SECURITY_JWT_JWK_SET_URI`	`http://localhost:9000/oauth2/jwks`	`https://auth.example/.../jwks`	JWKS Endpoint für Signaturprüfung.
`SECURITY_JWT_ISSUER`	*(leer)*	`https://auth.example/api/v1/auth`	Issuer-URI (Alternative zu JWKS URI; wenn gesetzt, wird Issuer validiert).
`SECURITY_JWT_AUDIENCE`	`template-service`	`template-service`	Erwartete Audience für `aud`-Check.
`SECURITY_JWT_CLOCK_SKEW_SECONDS`	`60`	`60`	Toleranz für Zeitvalidierung (`exp`).
`TEMPLATE_DEFAULT_LOCALE`	`en-GB`	`de-DE`	Default-Locale für Fallback.
`TEMPLATE_CACHE_TTL_SECONDS`	`120`	`300`	TTL für Template-Resolve Cache.
`TEMPLATE_EVENTS_ENABLED`	`false`	`true`	Aktiviert Domain Event Publishing (Logging Publisher).

Hinweis zur lokalen Docker-Umgebung (docker-compose):

- Der Service wird auf Port **8104** veröffentlicht.
- Die DB wird als MariaDB Container betrieben (Image-Konfiguration via `MYSQL_DATABASE`, `MYSQL_ROOT_PASSWORD`, `MYSQL_ALLOW_EMPTY_PASSWORD` im Compose).

8. Fehlerbehandlung, Logging und Header-Konventionen

8.1 ErrorDTO (Response-Format)

Fehlerantworten verwenden ein einheitliches Modell mit Feldern:

- `timestamp`
- `status`
- `errorCode`
- `message`
- `details` (Map)
- `path`
- `correlationId`

8.2 CorrelationId

Der Service verwendet folgende Header-Reihenfolge:

1. `X-Request-Id`
2. `X-Correlation-Id`
3. `traceparent` (TraceId)
4. sonst generierte UUID

8.3 Standard-Statuscodes (Auszug)

- 200: erfolgreich
- 201: erstellt
- 400: Request ungültig
- 401: nicht authentifiziert / Token ungültig
- 403: nicht autorisiert (Scope fehlt)
- 404: Ressource nicht sichtbar oder nicht vorhanden (Non-Leak)
- 409: Konflikt (z. B. Governance, Immutable-Version)
- 422: Template nicht verarbeitbar (z. B. fehlende Placeholder bei MissingKeyPolicy=FAIL)
- 500: interner Fehler

Typische `errorCode`:

- `INVALID_REQUEST`
- `UNAUTHENTICATED`
- `ACCESS_DENIED`
- `TEMPLATE_NOT_FOUND`
- `VERSION_NOT_FOUND`
- `MISSING_KEYS`
- `TEMPLATE_KEY_RESERVED`
- `TEMPLATE_SHADOWING_FORBIDDEN`
- `VERSION_IMMUTABLE`
- `UNPROCESSABLE_TEMPLATE`
- `INTERNAL_ERROR`

9. Nicht-funktionale Anforderungen

- Performance: Resolve/Render ist cache-gestützt; Cache-Keys sind tenant-scharf.
- Sicherheit: Keine sensiblen Daten (Tokens, DB Passwörter) im Log.
- Audit: `createdBy/updatedBy` stammen aus JWT `sub`.
- Stabilität: Flyway Migrationen sind Bestandteil des Deployments.
- Dokumentation: OpenAPI/Swagger ist vollständig und beschreibt Requests, Responses, Fehlercodes und Beispiele.

10. Abnahmekriterien (Testfälle)

1. **Security / JWT**

- Token ohne `tenant_id` wird abgewiesen.
- Token ohne passende `aud` wird abgewiesen.
- Requests ohne erforderlichen Scope werden abgewiesen.

2. **Tenant Isolation**

- TENANT-Objekte eines Tenants sind für einen anderen Tenant nicht lesbar, nicht renderbar, nicht listbar (Antwort 404).
- `GET /families` liefert nur GLOBAL + TENANT des aktuellen Tenants.

3. **Governance**

- TENANT kann kein `platform.*` oder `system.*` anlegen.
- TENANT kann keinen Business-Key anlegen, der eine vorhandene GLOBAL-Familie überlagert.

4. **Versionierung**

- Erstellen einer Version setzt Status DRAFT und erhöht `versionNo`.
- Approve setzt Zielversion APPROVED und vorherige APPROVED DEPRECATED.
- PUT auf Version liefert Konflikt (Immutable).

5. **Rendering**

- `/render` rendert nur APPROVED.
- MissingKeyPolicy=FAIL liefert 422 und `details.missingKeys`.
- Locale-Fallback arbeitet in der Reihenfolge exakt → Sprache → Default.

6. **Ops**

- Swagger UI ist erreichbar.
- Health Endpoint liefert 200.